HABS analysis

```
rm(list=ls())
gc()
installing.packages("dplyr")
#
# The following packages are masked from 'package:stats':
#
#     filter, lag
# The following objects are masked from 'package:base':
#
#     intersect, setdiff, setequal, union
library(dplyr)
#
# Attaching package: 'dplyr'
# The following objects are masked from 'package:stats':
#
#     filter, lag
# The following objects are masked from 'package:base':
#
#     intersect, setdiff, setequal, union
library(ggplot2)
# Warnings:
# * Failed to fill site-local.profile.
library(haven)
#
# Attaching package: 'haven'
# The following objects are masked from 'package:lubridate':
#
#     hms, stop_at
# The following object is masked from 'package:ggplot2':
#
#     zap
library(ggplot2)
library(survival)
library(survminer)
# This is version 0.4.10
# about it type 'help("KMunicate")' (in quotes) and enjoy.
library(survival)
library(survminer)
library(ggplot2)
library(dplyr)
#
# Attaching package: 'dplyr'
# The following objects are masked from 'package:stats':
#
#     filter, lag
# The following objects are masked from 'package:base':
#
#     intersect, setdiff, setequal, union
library(survival)
library(survminer)
library(ggplot2)
library(dplyr)
```

LONGITUDINAL ANALYSIS

Sanity check - ensuresure if you want to recalculate slopes. This might take ~15-30 min.

```
mydata_long = mydata[!is.nan(mydata$mean_cortical_thickness),]
mydata_long = mydata_long[!is.nan(mydata_long$mean_cortical_thickness),]

# Fit models for each region
lme_results = data.frame(NULL)
for (region in unique(mydata_long$region)) {
  cat("Processing region:", region, "\n")
  tryout = lme(mean_cortical_thickness ~ age + sex + mean_cortical_thickness_baseline + TBV,
               random = ~1|subjectID,
               data = mydata_long[mydata_long$region == region,],
               na.action = na.omit)
  tryout = data.frame(summary(tryout)$tTable[4:5,])
  tryout$region = rep(region, 2)
  tryout$fixed_effect = c("age", "sex")
  lme_results = rbind(lme_results, tryout)
}
colnames(lme_results) = c("beta", "SE", "DF", "t_stat", "p_uncorrected", "region", "fixed_effect")

# Calculate FDR corrected p-values
lme_results$p_FDR = NULL
for (eff in unique(lme_results$fixed_effect)) {
  print(length(p.adjust(lme_results[lme_results$fixed_effect == eff,]$p_uncorrected, method = "fdr")))
  lme_results[lme_results$fixed_effect == eff,]$p_FDR = p.adjust(lme_results[lme_results$fixed_effect == eff,]$p_uncorrected, method = "fdr")
}

# Add cortical thickness baseline for context
baseline = mydata_long[mydata_long$visit_code == 'bl', c('region', 'mean_cortical_thickness')]
colnames(baseline) = c('region', 'baseline_thickness')
lme_results = merge(lme_results, baseline)

# Add significance flags
lme_results$sig_uncorrected = ifelse(lme_results$p_uncorrected < 0.05, "Sig", "Non-Sig")
lme_results$sig_FDR = ifelse(lme_results$p_FDR < 0.05, "Sig", "Non-Sig")

# Calculate effect sizes and CI
lme_results$CI_lower = lme_results$beta - 1.96 * lme_results$SE
lme_results$CI_upper = lme_results$beta + 1.96 * lme_results$SE

# Save results
write.csv(lme_results, "/content/drive/MyDrive/ADNI_Longitudinal_Results/longitudinal_lme_results_with_baseline.csv", row.names = FALSE)

# Summary statistics
cat("\n=== SUMMARY STATISTICS ===\n")
cat("Total tests:", nrow(lme_results), "\n")
cat("Significant uncorrected:", sum(lme_results$sig_uncorrected == "Sig"), "\n")
cat("Significant FDR:", sum(lme_results$sig_FDR == "Sig"), "\n")
cat("\n=== BY FIXED EFFECT ===\n")
table(lme_results$fixed_effect, lme_results$sig_FDR)

# Top results
cat("\n=== TOP 10 AGE EFFECTS ===\n")
head(lme_results[lme_results$fixed_effect == "age",][order(lme_results[lme_results$fixed_effect == "age",]$p_FDR),], 10)

cat("\n=== TOP 10 SEX EFFECTS ===\n")
head(lme_results[lme_results$fixed_effect == "sex",][order(lme_results[lme_results$fixed_effect == "sex",]$p_FDR),], 10)

# Save baseline thickness for reference
write.csv(baseline, "/content/drive/MyDrive/ADNI_Longitudinal_Results/baseline_thickness.csv", row.names = FALSE)
```

```
Processing region: bankssts
Processing region: caudalanterior cingulate
Processing region: caudalmiddle frontal
Processing region: cuneus
Processing region: entorhinal
Processing region: fusiform
Processing region: inferior parietal
Processing region: inferior temporal
Processing region: isthmus cingulate
Processing region: lateral occipital
Processing region: lateral orbitofrontal
Processing region: lingual
Processing region: medial orbitofrontal
Processing region: middle temporal
Processing region: parahippocampal
Processing region: paracentral
Processing region: pars opercularis
Processing region: pars orbitalis
Processing region: pars triangularis
Processing region: pericalcarine
Processing region: postcentral
Processing region: posterior cingulate
Processing region: precentral
Processing region: precuneus
Processing region: rostral anterior cingulate
Processing region: rostral middle frontal
Processing region: superior frontal
Processing region: superior parietal
Processing region: superior temporal
Processing region: supramarginal
Processing region: frontal pole
Processing region: temporal pole
Processing region: transverse temporal
Processing region: insula
[1] 68
[1] 68
[1] 68
```

```
DD = DD.dropna()
DD.shape, DD.head()
DD = DD.drop_duplicates()
DD.shape, DD.head()

(3000,5,0,0,0
```

```
gcn_pos_ = [f'gcn{i}=cmdstan(chain=1)[f'gcn{i}_pos',
                                        dtype=int,
                                        sampling=True,
                                        inits=0,
                                        init_bound=1,
                                        lower=0,
                                        upper=2,
                                        seed=1,
                                        output_dir='./tmp',
                                        show_progress=False,
                                        show_console=False,
                                        refresh=1000,
                                        chains=1,
                                        data=data)' for i in range(1, 5)]
```

```
gcn_pos_ = [f'gcn{i}_pos' for i in range(1, 5)]
print(gcn_pos_)
```

```
['gcn1_pos', 'gcn2_pos', 'gcn3_pos', 'gcn4_pos']
```

```
BCG_data_df = pd.read_csv('data/BCG_data.csv', dtype=dtypes_pos=dtypes, sep=',', low_memory=False, usecols=['A1', 'A2', 'A3', 'A4', 'gcn1_pos', 'gcn2_pos', 'gcn3_pos', 'gcn4_pos'])
print(BCG_data_df.shape, BCG_data_df.dtypes, BCG_data_df.head(), BCG_data_df.tail(), BCG_data_df.describe(), BCG_data_df.info(), BCG_data_df.sample(n=5))
```

```
(2,) (2,) (2,) (
```